

Entregables tercer corte



Universidad
del Cauca

Vigilada Mineducación

Ingeniería de Software 2

Presentado por:

Nicole Burbano Solarte
Juan Martín Cerón Chates
John Alexander Ramirez
Kevin Yesid Castaño

Profesor:

Wilson Libardo Pantoja Yopez, Roberto Encarnacion Mosquera

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Ingeniería de Sistemas

Popayán, junio de 2026

Introducción

El presente documento describe la evolución y los resultados obtenidos durante la tercera iteración del proyecto de Ingeniería de Software II, desarrollado para la Red de Servicios Médicos Piedrazul. En esta fase se consolidó la arquitectura distribuida basada en microservicios implementada en iteraciones anteriores, incorporando nuevos requisitos funcionales orientados a mejorar la experiencia de los pacientes y optimizar los procesos administrativos relacionados con el agendamiento de citas médicas. Durante esta iteración se implementan funcionalidades de alto valor para el cliente, incluyendo el agendamiento autónomo de citas por parte de los pacientes, la configuración de parámetros de disponibilidad de médicos y terapeutas, la exportación de citas y el re-agendamiento con trazabilidad de cambios. Adicionalmente, se fortalecieron aspectos no funcionales relacionados con la seguridad, mediante la implementación de autenticación y autorización basada en JWT y Keycloak, así como la aplicación de arquitectura hexagonal y principios de Domain-Driven Design (DDD) para mejorar la mantenibilidad y evolución del sistema. De igual manera, este documento presenta las historias de usuario desarrolladas durante las tres iteraciones, los escenarios de calidad asociados a los atributos de modificabilidad y escalabilidad, la arquitectura del sistema utilizando el modelo C4, los patrones de diseño implementados, la planificación de los sprints y los enlaces al repositorio del proyecto. El objetivo es evidenciar cómo las decisiones arquitectónicas y de diseño adoptadas permiten cumplir los requerimientos del negocio, garantizando una solución segura, escalable y preparada para futuras evoluciones.

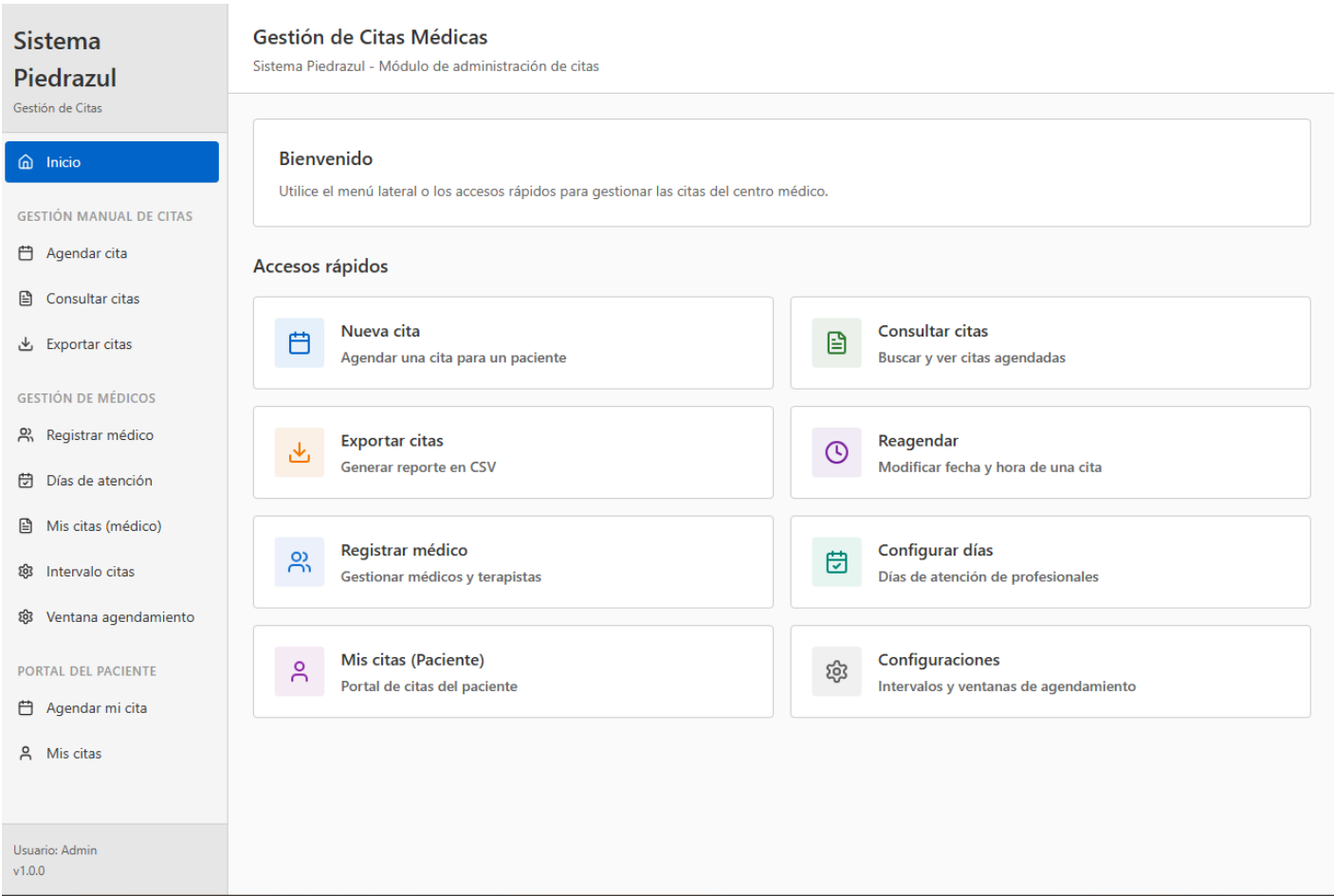
1. Historias de usuario (con criterios de aceptación) implementadas en la segunda iteración

[Excel de las épicas y sus historias de usuario](#)

2. Prototipos de la interfaz de usuario

Para el diseño de los prototipos de la interfaz de usuario se utilizó la herramienta Figma, la cual permitió crear representaciones visuales claras e intuitivas del sistema. Esto facilitó la definición de la experiencia de usuario y la validación temprana de las funcionalidades propuestas. En esta tercera iteración se presenta la configuración de disponibilidad de especialistas, re-agendamiento de citas y exportación de citas médicas, las cuales permiten optimizar la gestión de agendas, facilitar el seguimiento de los pacientes y mejorar los procesos operativos de la organización.

- **Gestión de médicos/terapistas:**



E2-HU1 (Registrar médicos/terapistas en el sistema):

Registrar Médico/Terapista

Gestione la información de los profesionales de la salud

Datos del profesional

Tipo de documento *

Seleccione...

▼

Número de documento *

Ej: 1234567890

Nombres *

Ej: Juan Carlos

Apellidos *

Ej: Gómez Pérez

Especialidad *

Seleccione...

▼

Celular *

Ej: 3001234567

Correo electrónico *

Ej: medico@piedrazul.com

Guardar

Limpiar

Volver

Listado de médicos/terapistas registrados

Documento	Nombre completo	Especialidad	Celular	Correo	Estado	Acción
CC 1234567890	Juan Carlos Gómez Pérez	Medicina General	3001234567	jgomez@piedrazul.com	Activo	Editar
CC 9876543210	María Elena Martínez López	Pediatría	3009876543	mmartinez@piedrazul.com	Activo	Editar

E2-HU2 (Configurar la franja horaria de atención de cada médico/terapista):

Configurar Días de Atención

Defina los días de la semana en que cada profesional atiende

Selección de profesional

Médico/Terapista *

Dr. Juan Gómez - Medicina General

Días de atención

Seleccione los días *

☒ Lunes

☐ Martes

☒ Miércoles

☐ Jueves

☒ Viernes

☐ Sábado

☐ Domingo

Configuración actual:
Lunes, Miércoles, Viernes

Guardar configuración

Volver

Mis Citas Programadas

Consulte sus citas como profesional de la salud

Filtros de búsqueda

Profesional

Dr. Juan Gómez - Medicina General

Fecha inicio

mm/dd/aaaa

Fecha fin

mm/dd/aaaa

Estado

Todas

Limpiar filtros

Volver

Citas encontradas (4)

Fecha	Hora	Paciente	Tipo de atención	Estado	Acción
27/3/2026	09:00 AM	Carlos Rodríguez	Consulta médica	Programada	Ver detalle
27/3/2026	10:30 AM	Ana María López	Control de seguimiento	Programada	Ver detalle
19/3/2026	02:00 PM	José Martínez	Consulta médica	Completada	Ver detalle
14/3/2026	11:00 AM	Laura Sánchez	Consulta médica	Pasada	Ver detalle

E2-HU4(Definir el intervalo de tiempo entre citas por médico/terapista):

Configurar Intervalo entre Citas

Defina el tiempo en minutos entre cada cita del profesional

Configuración del intervalo

Médico/Terapista *

Dr. Juan Gómez - Medicina General

Intervalo entre citas (minutos) *

30

Intervalo actual:

30 minutos

Nota:

Este intervalo determina el tiempo mínimo entre citas consecutivas. Por ejemplo, si configura 30 minutos y una cita es a las 09:00 AM, la siguiente cita disponible será a las 09:30 AM.

Guardar

Volver

Intervalos configurados

Profesional	Intervalo (minutos)
Dr. Juan Gómez - Medicina General	30 minutos
Dra. María Martínez - Pediatría	45 minutos

E2-HU5 (Configurar la ventana de tiempo (en semanas) para el agendamiento de citas):

Configurar Ventana de Agendamiento

Defina el rango de semanas futuras permitidas para agendar citas

Configuración de ventana temporal

Número de semanas para agendamiento *

8

Vista previa:

Los pacientes podrán agendar citas desde hoy hasta **8** semanas en el futuro.

¿Qué es la ventana de agendamiento?

Es el período máximo futuro en el que se permite agendar citas. Por ejemplo:

- Si configura **4 semanas**, los pacientes solo podrán agendar citas en los próximos 28 días.
- Si configura **8 semanas**, el rango será de 56 días.
- Esto ayuda a controlar la disponibilidad de citas a largo plazo.

Configuración actual: 8 semanas

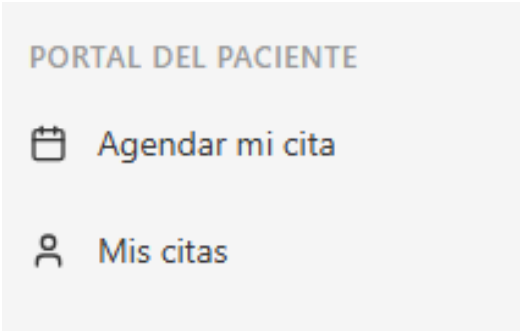
Guardar

Volver

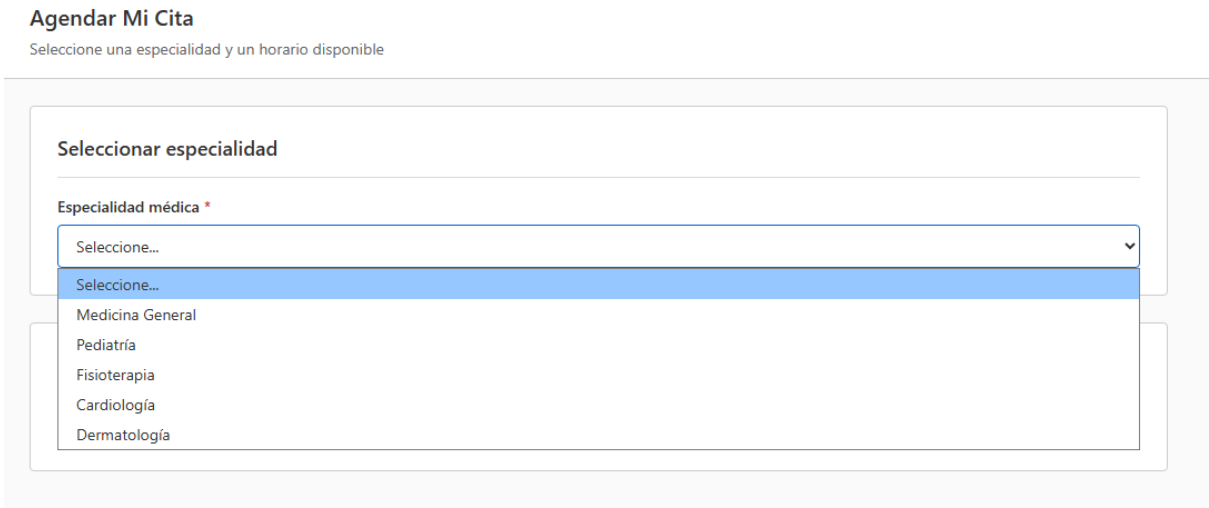
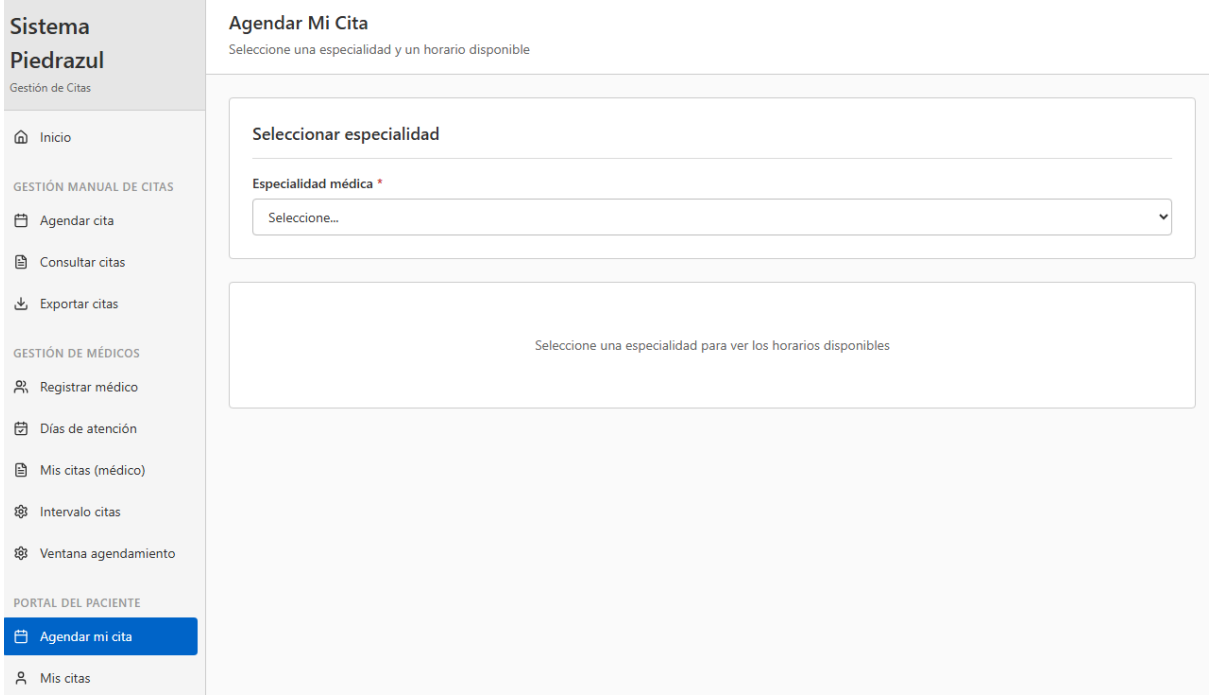
Información adicional

- Esta configuración es global y aplica para todos los profesionales y especialidades.
- El sistema bloqueará automáticamente fechas fuera del rango configurado.
- Se recomienda configurar un valor entre 2 y 8 semanas para un equilibrio adecuado.
- Los cambios en esta configuración aplican de inmediato en el módulo de agendamiento.

- **Gestión autónoma de citas**



E3-HU1(Agendar una cita como paciente):



Seleccionar especialidad

Especialidad médica *

Pediatría

Horarios disponibles

Estos horarios están disponibles según la disponibilidad del profesional, días festivos y horarios de atención.

viernes, 27 de marzo de 2026
09:00 AM - Dr. Juan Gómez

viernes, 27 de marzo de 2026
10:30 AM - Dr. Juan Gómez

sábado, 28 de marzo de 2026
08:00 AM - Dr. Juan Gómez

sábado, 28 de marzo de 2026
11:00 AM - Dra. María Martínez

domingo, 29 de marzo de 2026
09:30 AM - Dr. Carlos López

viernes, 27 de marzo de 2026
10:30 AM - Dr. Juan Gómez

sábado, 28 de marzo de 2026
08:00 AM - Dr. Juan Gómez

sábado, 28 de marzo de 2026
11:00 AM - Dra. María Martínez

domingo, 29 de marzo de 2026
09:30 AM - Dr. Carlos López

Resumen de su cita

Especialidad:

Pediatría

Profesional:

Dr. Juan Gómez

Fecha:

27/3/2026

Hora:

10:30 AM

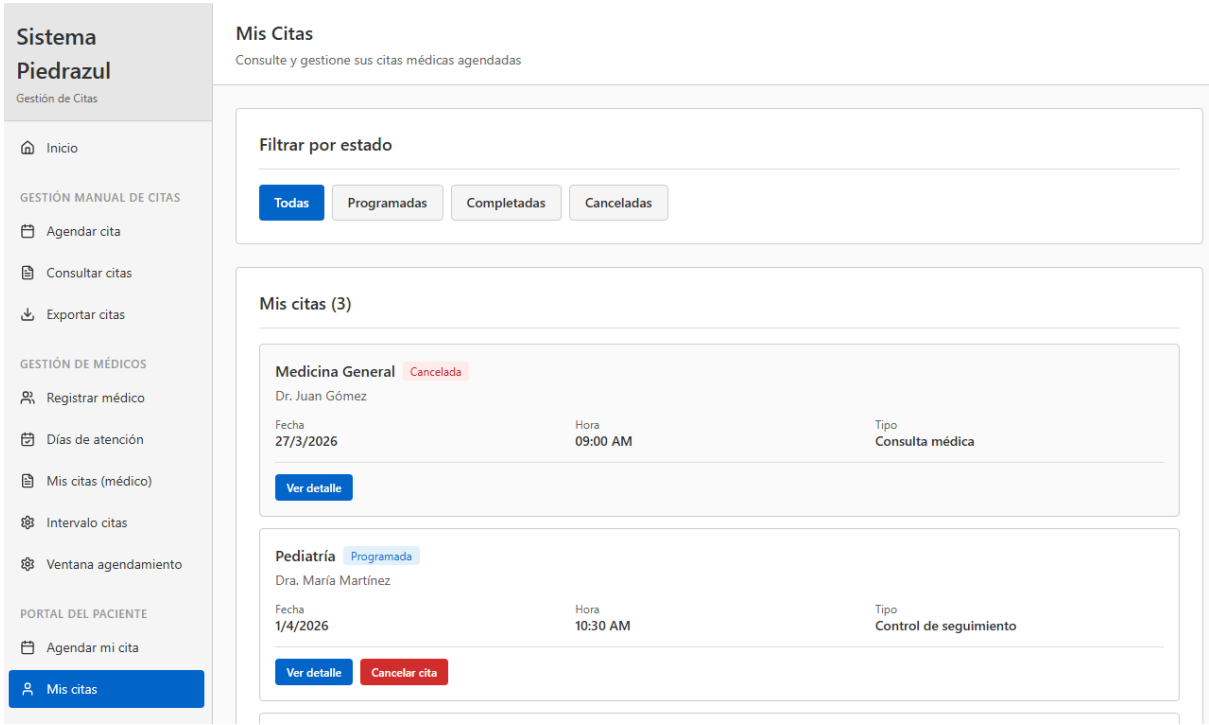
Agendar cita

Cancelar selección

E3-HU2(Cancelar cita como paciente):



E3-HU3(Consultar cita como paciente):



The image shows a modal window titled "Detalle de la cita" (Appointment Details) with a close button (X) in the top right corner. The modal contains several fields with labels and values:

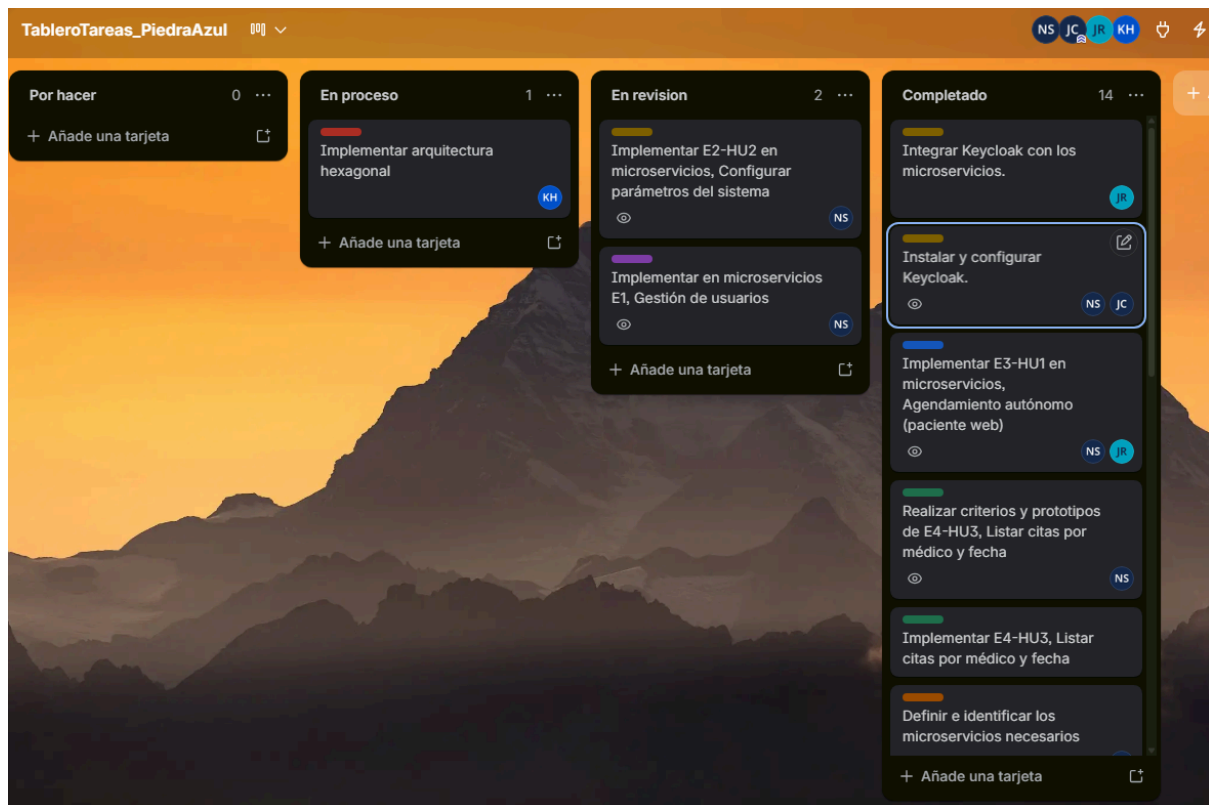
- Especialidad:** Pediatría
- Profesional:** Dra. María Martínez
- Fecha:** 1/4/2026
- Hora:** 10:30 AM
- Tipo de atención:** Control de seguimiento
- Estado:** Programada (highlighted in blue)

At the bottom right of the modal is a button labeled "Cerrar" (Close).

3. Pantallazo de planificación de tareas del Sprint 3

Para la gestión y seguimiento de las actividades correspondientes al Sprint 2 se utilizó la herramienta Trello, permitiendo organizar las tareas del equipo de desarrollo de manera colaborativa. En el tablero se distribuyeron las actividades en diferentes estados: "Por hacer", "En proceso", "En revisión" y "Completado", facilitando el control del avance de cada historia de usuario y las responsabilidades asignadas a los integrantes del equipo.

Durante este sprint se planificaron y ejecutaron tareas enfocadas en la consolidación de la arquitectura distribuida del sistema, incluyendo la implementación de la arquitectura hexagonal en los microservicios, la integración de mecanismos de seguridad mediante Keycloak y JWT, el desarrollo de funcionalidades de agendamiento autónomo para pacientes, la gestión de usuarios y roles, la configuración de parámetros de disponibilidad para médicos y terapeutas, y la preparación de la solución para su despliegue y documentación final.



4. Escenario de calidad: Modificabilidad

Contexto:

El sistema de reserva de citas de Piedrazul está implementado mediante una arquitectura de microservicios. El microservicio de citas utiliza arquitectura hexagonal y principios de Domain-Driven Design (DDD), permitiendo desacoplar la lógica de negocio de la infraestructura y de los mecanismos de persistencia.

Estímulo:

El cliente solicita incorporar un nuevo tipo de profesional de salud, denominado Nutricionista, para que los pacientes puedan agendar citas con este profesional utilizando los mismos mecanismos de reserva existentes.

Respuesta:

El equipo de desarrollo realiza los cambios únicamente en el dominio y en los adaptadores relacionados con el microservicio de citas, agregando las nuevas reglas de negocio y actualizando los puntos de integración necesarios, sin afectar los demás microservicios del sistema.

Medición de calidad:

La modificación requiere intervenir únicamente el microservicio de citas, modificando menos de 10 clases existentes y sin generar cambios en los servicios de autenticación, usuarios o configuración.

Resultado esperado:

El sistema permite gestionar citas para nutricionistas manteniendo el correcto

funcionamiento de las funcionalidades existentes y minimizando el esfuerzo de mantenimiento gracias al desacoplamiento proporcionado por la arquitectura hexagonal.

-Escenario de Calidad: Escalabilidad

Contexto:

El sistema se encuentra desplegado mediante una arquitectura basada en microservicios y contenedores Docker, con autenticación centralizada mediante Keycloak y servicios independientes para la gestión de usuarios, citas y configuraciones.

Estímulo:

Durante una campaña de salud promovida por Piedrazul, se presenta un incremento considerable en la cantidad de pacientes que intentan consultar disponibilidad y agendar citas de forma simultánea a través de la plataforma web.

Respuesta:

Se incrementa el número de instancias del microservicio de citas para distribuir la carga de trabajo, manteniendo operativos los demás servicios y sin necesidad de realizar modificaciones en el código fuente de la aplicación.

Medición de calidad:

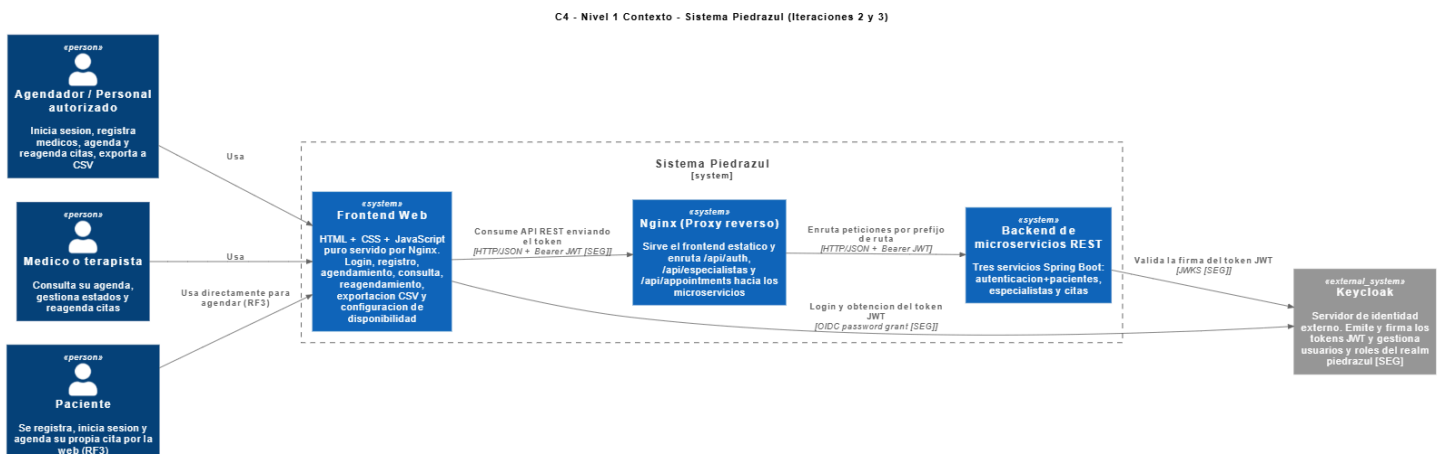
El sistema mantiene tiempos de respuesta inferiores a dos segundos para las operaciones de consulta y reserva de citas, incluso cuando la cantidad de solicitudes concurrentes aumenta significativamente respecto a la carga habitual.

Resultado esperado:

Los usuarios pueden continuar consultando y reservando citas sin interrupciones ni degradación perceptible del rendimiento, garantizando la disponibilidad y continuidad del servicio ante incrementos en la demanda.

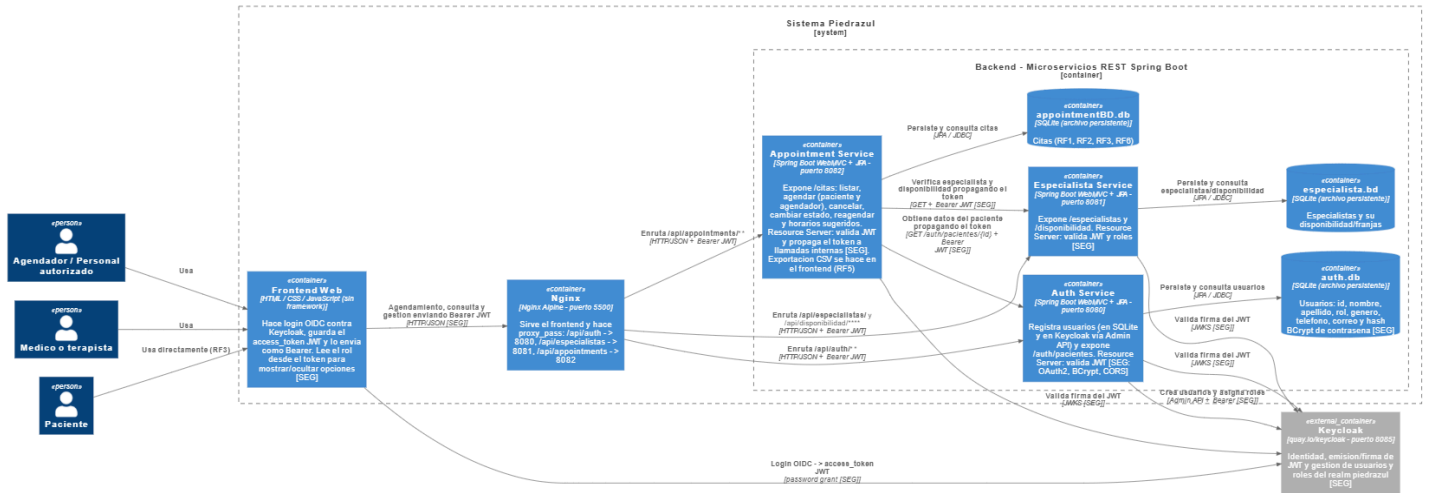
5. Arquitectura y diseño de software de la segunda iteración, usando el modelo C4

5.1 Diagrama de contexto



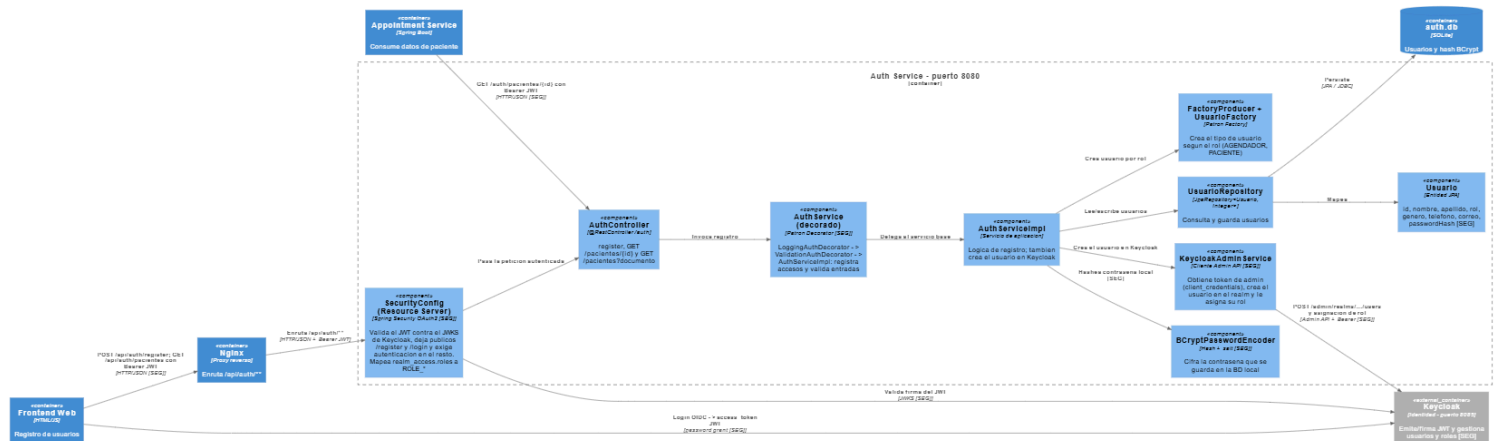
5.2 Diagrama de contenedores

C4 - Nivel 2 Contenedores - Sistema Piedrazul

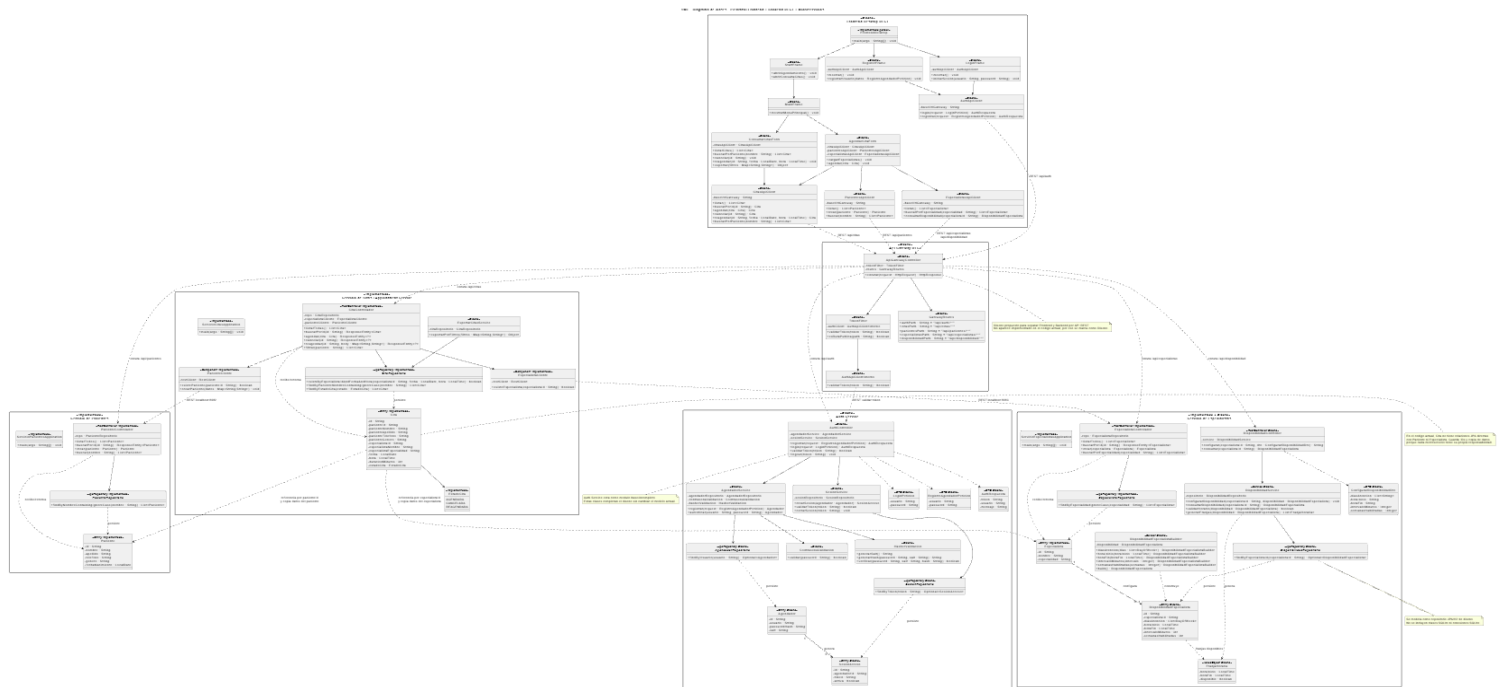


5.3 Diagrama de componentes

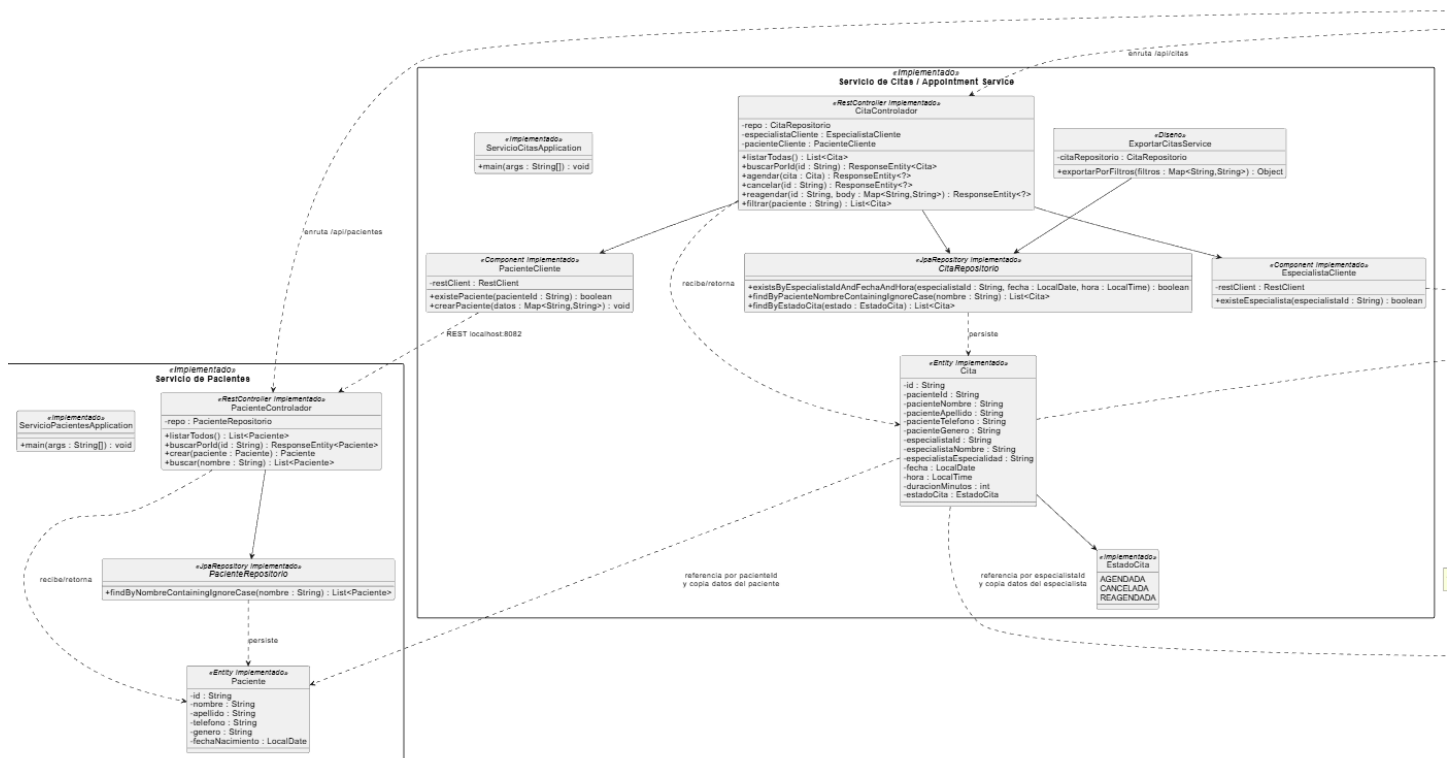
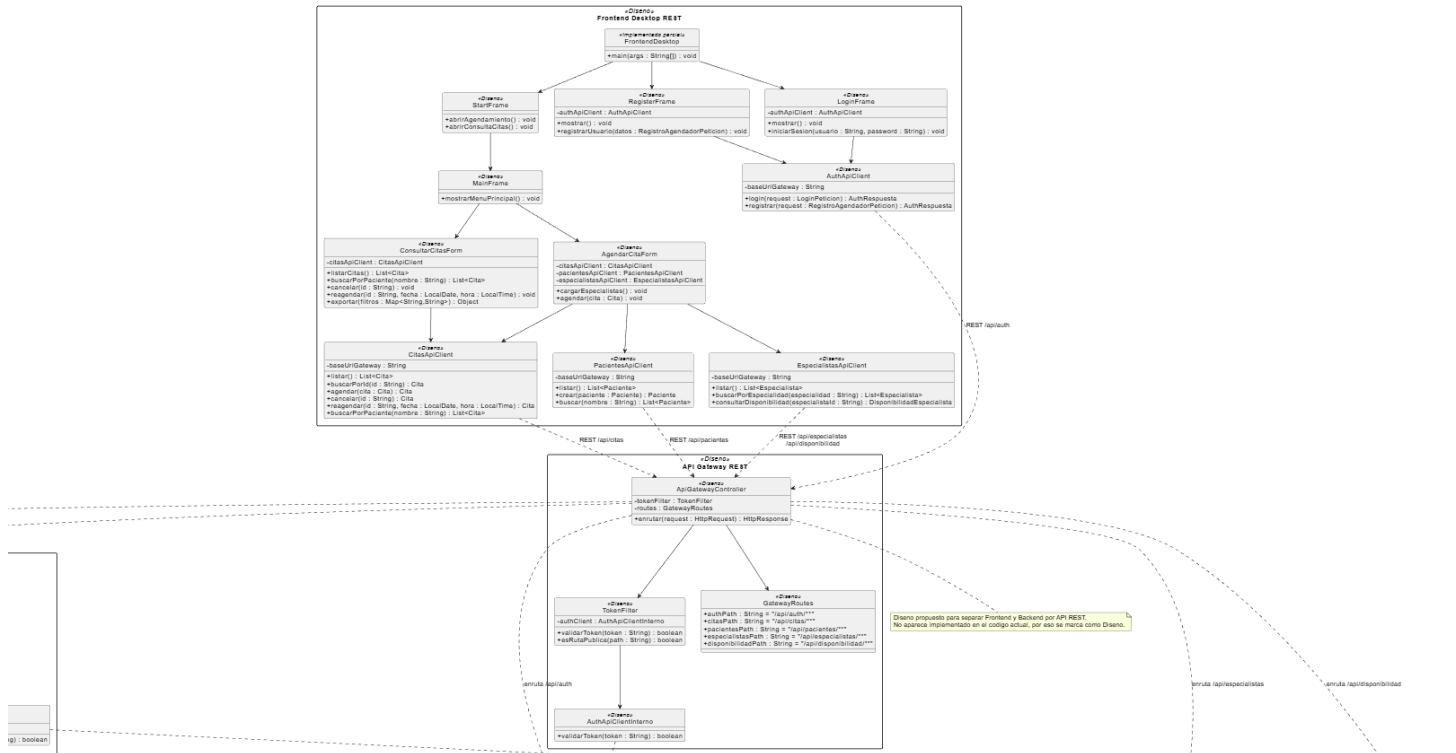
C4 - Nivel 3 Componentes - Auth Service (servicios de seguridad)

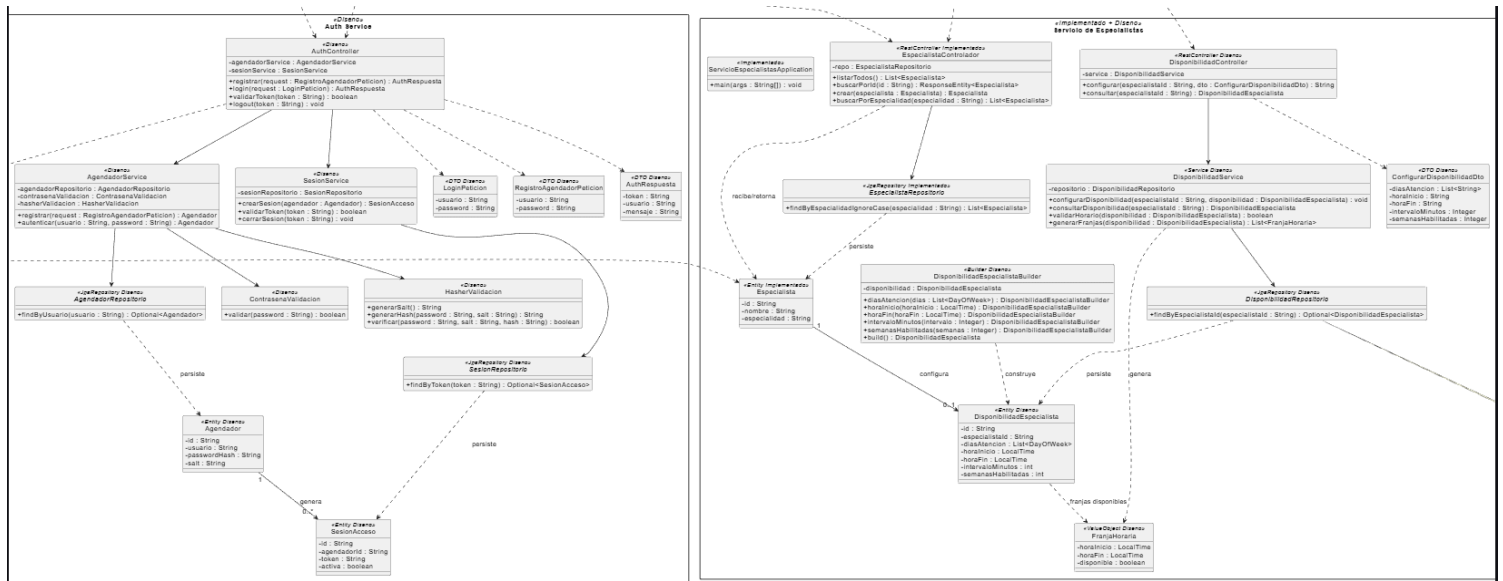


5.4 Diagrama de clases



UML - Diagrama de clases - Piedrazaul Frontend + Backend REST + Microservicios





(Mejor visualización):

Link: [Diagramas mas grandes](#)

6. Listado de patrones de diseño implementados y una breve descripción de cómo y en qué contexto del problema se aplicaron

1. Factory Method (+ Simple Factory) — auth-service, paquete *factory*

Se implementó el patrón Factory Method mediante la interfaz UsuarioFactory, encargada de definir la creación de usuarios, y las fábricas concretas PacienteFactory y AgendadorFactory, que construyen objetos específicos según el rol. La clase FactoryProducer selecciona dinámicamente la fábrica adecuada durante el proceso de registro en AuthServiceImpl. Este patrón se aplicó porque los distintos tipos de usuarios requieren atributos diferentes; por ejemplo, un paciente necesita información adicional como género, teléfono y fecha de nacimiento, mientras que un agendador no. De esta forma se encapsula la lógica de creación y se evita el uso de múltiples estructuras condicionales dentro del servicio de autenticación.

2. Decorator — auth-service, paquete *service*

El patrón Decorator fue utilizado para añadir funcionalidades adicionales al servicio de autenticación sin modificar su implementación principal. La interfaz AuthService actúa como componente base, AuthServiceImpl como componente concreto y las clases ValidationAuthDecorator y LoggingAuthDecorator como decoradores que agregan validaciones y registros de eventos respectivamente. La cadena de decoradores se configura en AuthConfig. Este patrón permitió incorporar validaciones de correo, contraseña, teléfono y fechas, así como mecanismos de auditoría y registro de operaciones de autenticación, manteniendo separadas las

responsabilidades y mejorando la mantenibilidad del código.

3. Builder — especialista-service, paquete *builder*

El patrón Builder fue implementado mediante la clase `DisponibilidadEspecialistaBuilder`, utilizada para construir objetos complejos relacionados con la configuración de disponibilidad de los especialistas. Este builder permite establecer de manera fluida atributos como días de atención, horario de inicio y fin, intervalo entre citas y número de semanas habilitadas para agendamiento. Su uso evita la creación de constructores extensos con numerosos parámetros y facilita la construcción de objetos de forma clara, legible y flexible dentro del proceso de configuración realizado desde el controlador.

4. Repository / DAO — especialista-service, auth-service y appointment-service

El patrón Repository se utilizó para abstraer el acceso a los datos y desacoplar la lógica de negocio de los mecanismos de persistencia. En el microservicio de especialistas se implementó mediante las interfaces `EspecialistaRepository` y `DisponibilidadRepository`, junto con implementaciones concretas basadas en JDBC y SQLite. En los microservicios de autenticación y citas se utilizó Spring Data JPA a través de repositorios como `UsuarioRepository` y `CitaRepository`. Este patrón permite que los servicios trabajen con abstracciones en lugar de depender directamente de consultas SQL o tecnologías específicas de almacenamiento, facilitando futuras modificaciones y pruebas.

5. DTO (Data Transfer Object) — todos los microservicios

El patrón DTO fue empleado para transportar información entre las diferentes capas de la aplicación y entre microservicios sin exponer directamente las entidades del dominio. Se implementó mediante clases como `RegisterRequest`, `LoginRequest`, `AuthResponse`, `PacienteResponse`, `CrearEspecialistaDto`, `ConfigurarDisponibilidadDto` y `AgendarPacienteRequest`. Este patrón permite definir contratos claros para las APIs REST, reducir el acoplamiento entre componentes y proteger la estructura interna de las entidades frente a cambios futuros en la implementación.

6. Proxy Remoto / Service Client (Gateway) — appointment-service, paquete *client*

El patrón Proxy Remoto se implementó mediante los clientes `EspecialistaClient`, `PacienteClient` y `DisponibilidadClient`, los cuales encapsulan las llamadas REST realizadas desde el microservicio de citas hacia otros microservicios del sistema. Estos clientes actúan como representantes locales de servicios remotos y permiten acceder a información relacionada con pacientes, especialistas y disponibilidad sin que la lógica de negocio conozca los detalles de la comunicación distribuida. Gracias a este patrón se logra un menor acoplamiento entre servicios y una mejor organización de las dependencias en la arquitectura de microservicios.

7. Interceptor — appointment-service, configuración REST

El patrón Interceptor fue utilizado para interceptar y procesar automáticamente las solicitudes realizadas entre microservicios. Su implementación se encuentra en la configuración REST del sistema, donde un interceptor agrega y propaga el token JWT en cada petición saliente. Este mecanismo garantiza que la información de autenticación y autorización viaje de manera transparente entre los diferentes servicios, evitando la duplicación de código en cada cliente REST y centralizando la gestión de la seguridad dentro de la aplicación.

7. URL del repositorio GIT

<https://github.com/juanmceron-arch/piedrazul-microservicios.git>